

Project Title
Task Management
System

Introduction to
Problem Solving
(CSE1021)

Name : Seru Mahesh
Reg n.o : 25MIP10040
Submitted to : Dr. Dheresh soni
Slot :A14+D11+D12

1.Introduction:

The To-Do List Application is a command-line project designed to help users efficiently manage daily tasks. Users can add tasks, set due dates, mark completion status, and delete tasks. The app ensures basic task organization and showcases core Python concepts such as loops, lists, dictionaries, functions, and date handling.

This project is ideal for beginners and demonstrates how Python can be used to build simple yet effective real-world applications.

2.Problem Statement:

People often forget important tasks or deadlines because they do not have a simple way to track and organize them. Many to-do list applications are too complex for beginners or require internet access.

There is a need for an easy, offline, and beginner-friendly task management system that runs on any device with Python installed.

3.Functional Requirements

1. Add a new task

Allows the user to enter a task along with its due date.

The task is stored in the task list for later retrieval.

2. View all tasks

Displays the complete list of tasks with their status and due dates.

Helps the user see all pending and completed tasks together.

3. Mark task as completed

Lets the user change a task's status from pending to completed.

Useful for tracking progress and what tasks are done.

4. Delete a task

Allows the user to remove any task from the list permanently.

Helps maintain a clean and updated task list.

5. View pending tasks only

Shows only the tasks that are not yet completed.

Makes it easier to focus on tasks that still need attention.

6. Exit the application

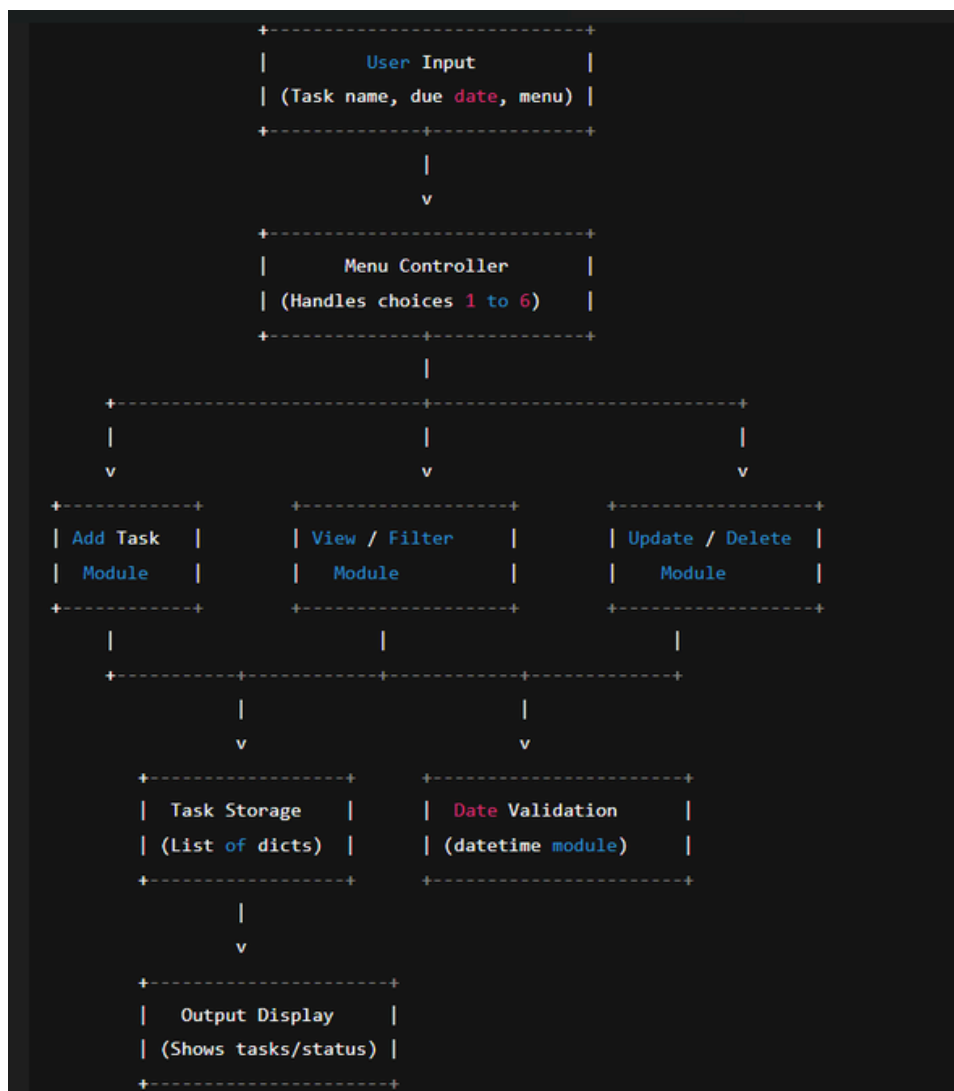
Closes the program safely without losing data during the session.

Provides a clean termination of the user workflow.

4.Non-functional Requirements

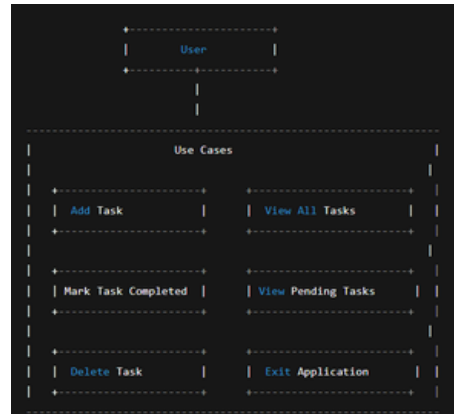
- **Usability:** Interface must be simple and text-based.
- **Reliability:** The system should not crash due to user errors.
- **Performance:** All operations must be fast and responsive.
- **Portability:** Should run on Windows, Linux, Android (with Pydroid/Termux).
- **Maintainability:** Code should be be modular and readable.
- **Scalability:** Easy to add new features in the future

5.System Architecture

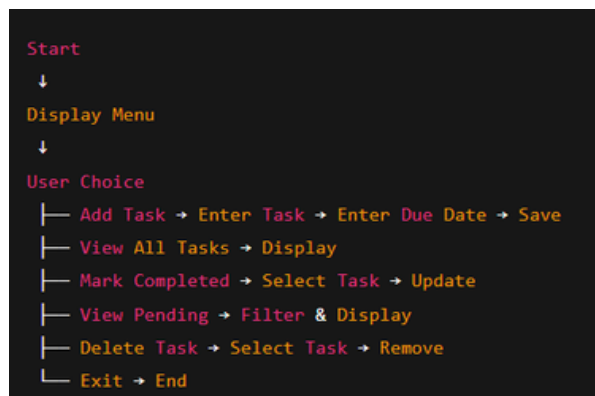


6.Design Diagram

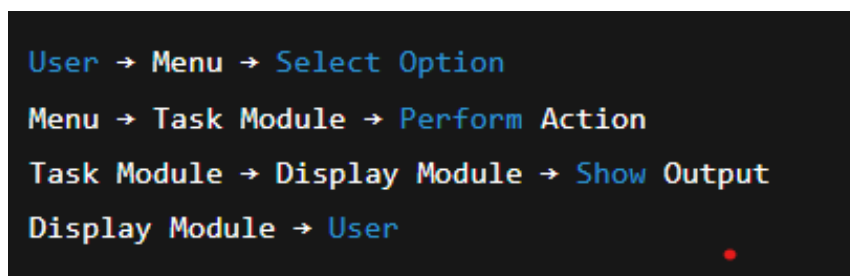
a)Use Case Diagram:



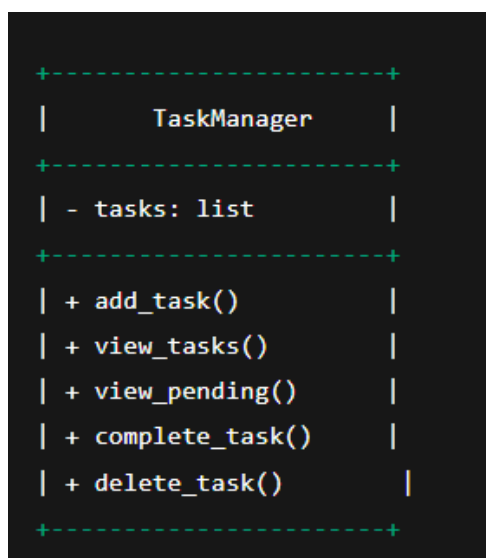
b)Workflow Diagram:



c)Sequence Diagram:



d)Class Diagram:



7.Design Decisions and Rationale

- **Use of Python**

Simple, powerful, easy for beginners

- **CLI interface**

Works everywhere, no GUI needed

- **Lists + dictionaries**

Best for storing structured task data

- **Due date input**

Makes tasks more realistic

- **Modular functions**

Easier to maintain and expand

8.Implementation Details

- **Language:** Python 3

- **Modules used:**

- datetime (for due dates)

- **Core Concepts Used:**

- Lists
- Dictionaries
- Loops
- Functions
- Conditionals
- File: todo.py

Each function handles a separate part of the application:

- add_task() → task creation
- view_tasks() → display
- complete_task() → mark complete
- delete_task() → remove task
- view_pending_tasks() → filter tasks

9.Screenshots

```
----- TO DO LIST MENU -----
1. Add a new task
2. View all tasks
3. Mark task as completed
4. Delete a task
5. View pending tasks only
6. Exit
Enter your choice (1-6): 1
Enter the task: finish python project
Enter the due date (YYYY-MM-DD): 2025-11-25
Task added successfully with due date!

----- TO DO LIST MENU -----
1. Add a new task
2. View all tasks
3. Mark task as completed
4. Delete a task
5. View pending tasks only
6. Exit
Enter your choice (1-6): 1
Enter the task: finish physics assinment
Enter the due date (YYYY-MM-DD): 2025-12-05
Task added successfully with due date!

----- TO DO LIST MENU -----
1. Add a new task
2. View all tasks
3. Mark task as completed
4. Delete a task
5. View pending tasks only
6. Exit
Enter your choice (1-6): 2

----- ALL TASKS -----
1. finish python project --> X Pending (Due: 2025-11-25)
2. finish physics assinment --> X Pending (Due: 2025-12-05)
```

```

----- TO DO LIST MENU -----
1. Add a new task
2. View all tasks
3. Mark task as completed
4. Delete a task
5. View pending tasks only
6. Exit
Enter your choice (1-6): 3

----- ALL TASKS -----
1. finish python project --> X Pending (Due: 2025-11-25)
2. finish physics assnment --> X Pending (Due: 2025-12-05)
Enter task number to mark as completed: 1
Task marked as completed!

----- TO DO LIST MENU -----
1. Add a new task
2. View all tasks
3. Mark task as completed
4. Delete a task
5. View pending tasks only
6. Exit
Enter your choice (1-6): 5

----- PENDING TASKS -----
1. finish physics assnment --> X Pending (Due: 2025-12-05)

```

10. Testing Approach

- **Functional Testing**

Check each menu option works correctly

- **Input Validation**

Test valid/invalid dates

- **Boundary Testing**

Empty list, first & last task

- **Error Handling**

Wrong menu input, invalid task number

11.Challenges Faced

- Ensuring correct date format
- Handling invalid inputs without crashing
- Managing a list of tasks dynamically
- Making the interface simple but effective
- Avoiding duplicate code by using functions

12. Learnings & Key Takeaways

- How to create modular Python programs
- Usage of lists and dictionaries
- Input handling and validation
- Basics of software development lifecycle
- How to design a simple text-based system
- Understanding due dates with datetime

13. Future Enhancements

- Save tasks permanently using file storage
- Load tasks when program starts
- Add notifications or alerts for overdue tasks
- Add priority levels (High/Medium/Low)
- Implement a GUI using Tkinter
- Add color-coded terminal output
- Cloud sync and user login (advanced)

14. References

- Python Official Documentation: <https://docs.python.org/>
-
- W3Schools Python: <https://www.w3schools.com/python/>
-
- GeekForGeeks Python: <https://www.geeksforgeeks.org/python/>
-
- Project design ideas adapted from standard SDLC practices